Module 5: Generic Programming and the Java Collections Framework

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai Campus

premanand.s@vit.ac.in

March 30, 2026

Module 5: Syllabus Overview

Module Title

Generic Programming and the Java Collections Framework

Topics Covered:

- Generic Classes and Methods
- Collection Framework
- List and Map
- ArrayList and LinkedList
- TreeMap and HashMap
- Lambda Function
- Java Database Connectivity (JDBC)
- JDBC Architecture
- CRUD Operations with Databases

Why This Module?

Hook

Can real-world software applications work efficiently using only variables and arrays?

Why This Module?

Hook

Can real-world software applications work efficiently using only variables and arrays?

The answer is No.

This module is introduced to help us:

- write reusable and type-safe code
- store and manage dynamic data
- organize data efficiently
- connect Java with databases
- perform real-world CRUD operations

1. What is Data?

Definition

Data is any information that a program stores, processes, or manipulates.

Examples:

- Student name
- Roll number
- Marks
- Product price
- Mobile number

2. How Do We Store Data in Java?

Initially, Java provides:

- Variables
- Arrays

Example of Variables:

```
int age = 20;
String name = "Prem";
```

Problem: Variables are suitable only for storing **one value at a time**.

3. Arrays for Multiple Data

Example:

```
String[] names = {"Asha", "Ravi", "Kiran",
    "Meena"};
```

Advantage:

- Stores multiple values together

But Arrays have limitations...

4. Limitations of Arrays

1. **Fixed Size**

```
int[] marks = new int[3];
```

2. **Insertion / Deletion is difficult**
3. **No rich built-in methods**
   - difficult searching
   - difficult sorting
   - difficult dynamic handling

Conclusion

Arrays are useful, but not enough for dynamic real-world applications.

5. What is a Collection?

Definition

A **Collection** is a group of objects stored together.

Real-life Examples:

- Collection of books in a library
- Collection of songs in a playlist
- Collection of contacts in a phone
- Collection of students in a class

6. What is a Framework?

Definition

A **Framework** is a ready-made structure or organized system that helps us solve problems efficiently.

Real-life Analogy:

- House construction framework
- College system with rules and structure

In Programming:

- A set of pre-built classes, interfaces, and methods

7. What is Collection Framework?

Definition

The **Java Collection Framework (JCF)** is a set of classes and interfaces used to store and manipulate groups of objects dynamically.

In Simple Words:

- Collection = Group of objects
- Framework = Ready-made system

Collection Framework = Ready-made system to manage groups of obj

8. Why Do We Need Collection Framework?

Because real-world data is:

- Large
- Dynamic
- Frequently changing
- Often inserted, deleted, searched, and sorted

Collection Framework helps us to:

- Store data dynamically
- Insert and delete easily
- Search efficiently
- Sort data easily
- Organize data in useful forms

9. Array vs Collection

Feature	**Array**	**Collection**
Size	Fixed	Dynamic
Insertion/Deletion	Difficult	Easy
Built-in Methods	Limited	Many
Flexibility	Less	High
Real-world Suitability	Low	High

10. Applications of Collection Framework

Real-World Applications:

- Student Management System
- Banking System
- E-Commerce Applications
- Hospital Management
- Social Media Platforms
- Mobile Contact and Playlist Storage

11. Components of Collection Framework

Main Components:

1. **Interfaces**
   - Collection
   - List
   - Set
   - Queue
   - Map
2. **Classes**
   - ArrayList
   - LinkedList
   - HashMap
   - TreeMap
3. **Methods**
   - add(), remove(), get(), put(), size(), contains()

12. Basic Hierarchy of Collection Framework

Iterable

↓

Collection

↓

List Set Queue

Note:

- `Map` is part of the Collections Framework
- But `Map` is **not** a child of `Collection`

13. Topics in Your Syllabus

You mainly need to study:

- List
- Map
- ArrayList
- LinkedList
- HashMap
- TreeMap
- Lambda Function
- Generics (basic usage in collections)

Example: ArrayList

```
import java.util.ArrayList;

public class Demo {
  public static void main(String[] args) {
    ArrayList<String> names = new ArrayList<>();

    names.add("Asha");
    names.add("Ravi");
    names.add("Kiran");

    System.out.println(names);
  }
}
```

15. Understanding the Syntax

```
ArrayList<String> names = new ArrayList<>();
```

Breakdown:

- `ArrayList` → Collection class
- `<String>` → Type of data stored
- `names` → Variable name
- `new ArrayList<>()` → Object creation

16. Simple Problem Statement

Problem

A teacher wants to store the names of students present in the class.

Without Collection:

```
String s1 = "Anu";
String s2 = "Ravi";
String s3 = "Meena";
```

With Collection:

```
ArrayList<String> students = new ArrayList<>();

students.add("Anu");
students.add("Ravi");
students.add("Meena");
```

17. Why is Collection Better Here?

Using Collection, we can:

- Add more students easily
- Remove absent students
- Search a student name
- Count total students
- Display all students

Next Topic

Next: **List**

1. Why Are We Learning List?

We already know:

- Variables store one value
- Arrays store multiple values

But arrays have limitations:

- Fixed size
- Difficult insertion/deletion
- Less flexible for real-time applications

Need

So Java provides a more flexible way to store multiple elements:

List

2. What is a List?

Definition

A **List** is an **ordered collection** of elements in Java.

Simple Meaning:

- It stores multiple values
- It keeps the insertion order
- It allows duplicate values
- Elements can be accessed by index

3. Real-Life Analogy of List

Think of a **class attendance register**.

Example:

1. Anu
2. Ravi
3. Meena
4. Kiran

Why is this like a List?

- Order is maintained
- Same name can appear more than once
- Each student has a position/index

4. Official Definition of List

Java Definition

`List` is an interface in Java that is used to store an ordered collection of elements.

Important Points:

- `List` is an **interface**, not a class
- It defines rules for storing ordered data
- Classes like `ArrayList` and `LinkedList` implement it

5. Why Do We Need List?

We need List when:

- We want to store multiple values dynamically
- We want to preserve insertion order
- We want to allow duplicate values
- We want to access elements using index

Examples:

- Student names in a class
- Shopping cart items
- Playlist songs
- Product names
- To-do tasks

6. What Problem Does List Solve?

Problem with arrays:

- Fixed size
- Hard to manage when data changes

List solves this by providing:

- Dynamic size
- Easy insertion
- Easy deletion
- Built-in methods
- Better flexibility

7. Features of List

1. Maintains insertion order
2. Allows duplicate values
3. Allows index-based access
4. Dynamic in size
5. Stores objects
6. Can store null values

8. Example of List

```
["Anu", "Ravi", "Meena", "Ravi"]
```

Observe:

- Order is maintained
- Duplicate value (`"Ravi"`) is allowed
- Each element has an index

Indexes:

0, 1, 2, 3

9. List vs Array

Feature	**Array**	**List**
Size	Fixed	Dynamic
Duplicates	Allowed	Allowed
Order	Maintained	Maintained
Built-in Methods	Few	Many
Insertion/Deletion	Difficult	Easy

10. Where is List Used?

Real-World Applications of List:

- Student attendance list
- Product catalog
- Mobile contacts
- Movie playlist
- Chat message history
- Online shopping cart

11. Syntax of List

General Syntax:

```
List<DataType> variableName = new ClassName<>();
```

Example:

```
List<String> names = new ArrayList<>();
```

Breakdown:

- `List` → interface
- `<String>` → type of elements
- `names` → variable name
- `ArrayList` → implementation class

12. Why Write List Instead of ArrayList?

Recommended Style:

```
List<String> names = new ArrayList<>();
```

Why?

- More flexible
- Easy to switch implementation later
- Follows good Java programming practice

Example: Today:

```
List<String> names = new ArrayList<>();
```

Tomorrow:

```
List<String> names = new LinkedList<>();
```

13. Common Methods of List

Important List methods:

- `add()`
- `get()`
- `set()`
- `remove()`
- `size()`
- `contains()`
- `clear()`
- `isEmpty()`

14. add() Method

Purpose

Used to insert an element into the List

Example:

```
List<String> names = new ArrayList<>();

names.add("Anu");
names.add("Ravi");
names.add("Meena");
```

Output Conceptually:

```
["Anu", "Ravi", "Meena"]
```

15. get() Method

Purpose

Used to access an element by index

```
System.out.println(names.get(1));
```

Output:

```
Ravi
```

Important: Index starts from 0

16. set() Method

Purpose

Used to update/replace an existing element

```
names.set(1, "Kiran");
```

Before:

```
["Anu", "Ravi", "Meena"]
```

After:

```
["Anu", "Kiran", "Meena"]
```

17. remove() Method

Purpose

Used to remove an element

By index:

```
names.remove(1);
```

By value:

```
names.remove("Anu");
```

18. size(), contains(), clear(), isEmpty()

Examples:

```
System.out.println(names.size());
System.out.println(names.contains("Ravi"));
names.clear();
System.out.println(names.isEmpty());
```

Use:

- `size()` → total number of elements
- `contains()` → checks existence
- `clear()` → removes all elements
- `isEmpty()` → checks if list is empty

19. Full Basic Program on List

```
import java.util.*;

public class Demo {
  public static void main(String[] args) {
    List<String> students = new ArrayList<>();

    students.add("Anu");
    students.add("Ravi");
    students.add("Meena");

    System.out.println(students);
    System.out.println(students.get(1));
    students.set(1, "Kiran");
    System.out.println(students);
    students.remove("Anu");
    System.out.println(students);
    System.out.println(students.size());
  }
}
```

20. Beginner Level Problem 1

Problem

Store 5 student names in a List and print them.

Expected Skills:

- Create List
- Add elements
- Display List

21. Beginner Level Problem 2

Problem

Store 5 marks in a List and print the total number of marks entered.

Expected Skills:

- Use `List<Integer>`
- Use `add()`
- Use `size()`

22. Intermediate Level Problem 1

Problem

Store 5 fruits in a List. Remove one fruit and display the updated List.

Expected Skills:

- Add elements
- Remove an element
- Print updated list

23. Intermediate Level Problem 2

Problem

Store 5 products in a List. Check whether "Laptop" exists in the list.

Expected Skills:

- Use `contains()`
- Logical checking

24. HOTS Problem 1

Problem

Create a student attendance manager using List.

Requirements:

- Add students
- Remove absent students
- Display all present students
- Display total count

25. HOTS Problem 2

Problem

Create an online shopping cart using List.

Requirements:

- Add items to cart
- Remove an item
- Search for an item
- Display all items

26. Common Viva / Exam Questions

1. What is a List in Java?
2. Why do we need List?
3. What are the features of List?
4. Difference between Array and List?
5. Can List store duplicate values?
6. Can List maintain insertion order?
7. Is List a class or interface?
8. Name any two classes that implement List

27. Key Takeaway

Final Understanding

A **List** is used in Java when we want to store multiple values in an **ordered, dynamic, and flexible** way.

Remember:

- Order is maintained
- Duplicates are allowed
- Index-based access is possible
- It is better than arrays for dynamic applications

Next: **ArrayList Vs LinkedList**

1. Common Point

Important

Both **ArrayList** and **LinkedList**:

- Implement the `List` interface
- Maintain insertion order
- Allow duplicate elements
- Support index-based operations

2. Key Difference Idea

Core Difference

- **ArrayList** uses a **dynamic array**
- **LinkedList** uses a **doubly linked list**

Meaning:

- ArrayList stores elements in contiguous memory
- LinkedList stores elements as nodes connected via links

3. Internal Structure

ArrayList:

- Continuous memory (array-based)
- Faster access using index

LinkedList:

- Nodes connected by pointers
- Each node stores:
  - data
  - previous reference
  - next reference

4. Comparison Table

Feature	**ArrayList**	**LinkedList**
Structure	Dynamic Array	Doubly Linked List
Access Time	Fast (O(1))	Slow (O(n))
Insertion	Slow (shifting needed)	Fast
Deletion	Slow (shifting needed)	Fast
Memory Usage	Less	More

5. Performance Understanding

ArrayList is better when:

- Frequent data access is required
- Index-based operations are common

LinkedList is better when:

- Frequent insertion/deletion is required
- Data changes dynamically at many positions

6. Example Scenario

Use ArrayList:

- Student marks access by index
- Reading data frequently

Use LinkedList:

- Queue operations
- Adding/removing elements frequently

7. Code Flexibility (List Interface)

Best Practice

Always program using the `List` interface:

```
List<String> list = new ArrayList<>();
```

```
List<String> list = new LinkedList<>();
```

Advantage:

- Easy to switch implementations
- Improves flexibility and maintainability

8. Key Exam Points

- Both implement `List`
- ArrayList uses dynamic array
- LinkedList uses node-based structure
- ArrayList is faster for access
- LinkedList is faster for insertion/deletion

9. Final Conclusion

Summary

- Use **ArrayList** for fast retrieval
- Use **LinkedList** for frequent updates

Golden Rule:

Access-heavy $\Rightarrow$ ArrayList | Modification-heavy $\Rightarrow$ LinkedList

Next: **LinkedList**

1. Why Are We Learning LinkedList?

We already learned:

- What a **List** is
- What an **ArrayList** is

Now we need another important implementation of `List`.

Need

One more major class that implements List is:

LinkedList

2. What is LinkedList?

Definition

LinkedList is a class in Java that implements the `List` interface and stores elements using a **linked node structure**.

Simple Meaning:

- It stores multiple values
- It maintains insertion order
- It allows duplicates
- It is very useful when insertion and deletion are frequent

3. Why is it Called LinkedList?

Because each element is stored in a **node**, and every node is **linked** to the next node.

Conceptually:

$$\boxed{\texttt{Anu}} \rightarrow \boxed{\texttt{Ravi}} \rightarrow \boxed{\texttt{Meena}}$$

So:

LinkedList = Elements connected like a chain

4. Why Do We Need LinkedList?

We need **LinkedList** when:

- We frequently insert elements
- We frequently delete elements
- Data changes often
- Order must be maintained

Examples:

- Waiting queue
- Music playlist
- Browser history
- Train coach sequence

5. What Problem Does LinkedList Solve?

Problem with ArrayList:

- Insertion/deletion in middle may be costly
- Shifting of elements may be required

LinkedList solves this by:

- Connecting elements through nodes
- Making insertion/deletion easier in many cases

6. Features of LinkedList

1. Maintains insertion order
2. Allows duplicate values
3. Dynamic in size
4. Supports index-based access
5. Better for insertion/deletion
6. Stores objects
7. Allows null values

7. Real-Life Analogy of LinkedList

Think of a **train**.

$$\boxed{\texttt{Coach 1}} \leftrightarrow \boxed{\texttt{Coach 2}} \leftrightarrow \boxed{\texttt{Coach 3}}$$

Observation:

- Each coach is connected to another coach
- One coach can be inserted or removed
- Structure still remains connected

8. Syntax of LinkedList

General Syntax:

```
LinkedList<DataType> variableName = new
    LinkedList<>();
```

Recommended Syntax:

```
List<String> names = new LinkedList<>();
```

Example:

```
LinkedList<String> names = new LinkedList<>();
```

9. Understanding the Syntax

```
LinkedList<String> names = new LinkedList<>();
```

Breakdown:

- `LinkedList` → class
- `<String>` → type of data
- `names` → variable name
- `new LinkedList<>()` → object creation

10. Import Statement

To use **LinkedList**, we must import:

```
import java.util.LinkedList;
```

or

```
import java.util.*;
```

11. Creating a LinkedList

```
import java.util.*;

public class Demo {
  public static void main(String[] args) {
    LinkedList<String> names = new LinkedList<>();
  }
}
```

Meaning: A dynamic linked structure is created to store String values.

12. add() Method

Purpose

Used to insert elements into the LinkedList

```
names.add("Anu");
names.add("Ravi");
names.add("Meena");
```

Result:

```
["Anu", "Ravi", "Meena"]
```

13. get() Method

Purpose

Used to access an element by index

```
System.out.println(names.get(1));
```

Output:

```
Ravi
```

14. set() Method

Purpose

Used to update an existing element

```
names.set(1, "Kiran");
```

Before:

```
["Anu", "Ravi", "Meena"]
```

After:

```
["Anu", "Kiran", "Meena"]
```

15. remove() Method

Purpose

Used to remove an element

By index:

```
names.remove(1);
```

By value:

```
names.remove("Anu");
```

16. Special Methods in LinkedList

LinkedList provides extra methods:

```
addFirst()
addLast()
getFirst()
getLast()
removeFirst()
removeLast()
```

Why important?

- Useful in queue-like and sequence-based operations

17. Example of Special Methods

```
LinkedList<String> names = new LinkedList<>();

names.add("Ravi");
names.add("Meena");

names.addFirst("Anu");
names.addLast("Kiran");

System.out.println(names);
```

Output:

```
[Anu, Ravi, Meena, Kiran]
```

18. size(), contains(), clear(), isEmpty()

Examples:

```
System.out.println(names.size());
System.out.println(names.contains("Ravi"));
names.clear();
System.out.println(names.isEmpty());
```

Use:

- `size()` → total elements
- `contains()` → checks existence
- `clear()` → removes all elements
- `isEmpty()` → checks whether list is empty

19. Full Basic Program on LinkedList

```
import java.util.*;

public class Demo {
  public static void main(String[] args) {
    LinkedList<String> students = new
       LinkedList<>();

    students.add("Anu");
    students.add("Ravi");
    students.add("Meena");
    System.out.println(students);
    System.out.println(students.get(1));
    students.set(1, "Kiran");
    System.out.println(students);
    students.addFirst("Divya");
    students.addLast("Saran");
    System.out.println(students);
    students.remove("Anu");
    System.out.println(students);}
}
```

20. Output of the Program

```
[Anu, Ravi, Meena]
Ravi
[Anu, Kiran, Meena]
[Divya, Anu, Kiran, Meena, Saran]
[Divya, Kiran, Meena, Saran]
```

21. Where is LinkedList Used?

Real-World Applications:

- Browser back/forward history
- Music playlist navigation
- Train coach connection model
- Waiting queue systems
- Undo / redo operations

22. Beginner Level Problem 1

Problem

Store 5 student names in a LinkedList and print them.

Expected Skills:

- Create LinkedList
- Use `add()`
- Display list

23. Beginner Level Problem 2

Problem

Store 4 fruits in a LinkedList. Add one fruit at the beginning and one at the end.

Expected Skills:

- Use `addFirst()`
- Use `addLast()`

24. Intermediate Level Problem 1

Problem

Store 5 tasks in a LinkedList. Remove the first and last task.

Expected Skills:

- Use `removeFirst()`
- Use `removeLast()`

25. Intermediate Level Problem 2

Problem

Store 5 songs in a LinkedList. Display the first and last song.

Expected Skills:

- Use `getFirst()`
- Use `getLast()`

26. HOTS Problem 1

Problem

Create a Browser History Manager using LinkedList.

Requirements:

- Add visited pages
- Display first visited page
- Display latest visited page
- Remove last visited page

27. HOTS Problem 2

Problem

Create a Train Coach Sequence using LinkedList.

Requirements:

- Add coach at beginning
- Add coach at end
- Remove a coach
- Display all coaches

28. Common Viva / Exam Questions

1. What is LinkedList in Java?
2. Why do we need LinkedList?
3. What are the features of LinkedList?
4. Why is it called LinkedList?
5. Can LinkedList store duplicate values?
6. Can LinkedList grow dynamically?
7. Is LinkedList a class or interface?
8. Which interface does LinkedList implement?
9. Name some special methods of LinkedList

29. Key Takeaway

Final Understanding

LinkedList is a powerful implementation of the `List` interface used when data needs **frequent insertion and deletion**.

Remember:

- Dynamic size
- Order maintained
- Duplicates allowed
- Strong for insertion/deletion operations

Next Topic

Next: **Map**

2. What is a Map?

Definition

A **Map** is a data structure in Java that stores elements in the form of **key-value pairs**.

Simple Meaning:

- Every value is connected to a key
- Key is used to identify the value
- Similar to dictionary / phone contacts

3. Real-Life Analogy of Map

Think of a **dictionary**.

Example:

- Apple $\rightarrow$ A fruit
- Java $\rightarrow$ A programming language
- Teacher $\rightarrow$ A person who teaches

Why is this like a Map?

- Left side = **Key**
- Right side = **Value**

4. Another Real-Life Analogy

Think of **mobile contacts**.

Example:

- Anu $\rightarrow$ 9876543210
- Ravi $\rightarrow$ 9123456780
- Meena $\rightarrow$ 9988776655

Observation:

- Name = Key
- Phone Number = Value

5. Official Definition of Map

Java Definition

`Map` is an interface in Java used to store data in **key-value pair** format.

Important Points:

- `Map` is an **interface**, not a class
- It is part of the **Collections Framework**
- But it is **not** a child of `Collection` interface

6. Why Do We Need Map?

We need Map when:

- One value must be linked to another value
- We want to search quickly using a key
- We want unique identification

Examples:

- Roll Number $\rightarrow$ Student Name
- Employee ID $\rightarrow$ Salary
- Product ID $\rightarrow$ Product Name
- Username $\rightarrow$ Password

7. What Problem Does Map Solve?

Problem: If we store only values in a List, searching becomes difficult.
Example:

- How to directly find student with roll number 102?
- How to directly find product with product ID 501?

Map solves this by:

- Connecting a **key** with a **value**
- Making search more meaningful and faster

8. Key and Value

Basic Structure

$$\texttt{Key} \rightarrow \texttt{Value}$$

Examples:

- 101 $\rightarrow$ Ravi
- 102 $\rightarrow$ Anu
- 103 $\rightarrow$ Kiran

Here:

- 101, 102, 103 = Keys
- Ravi, Anu, Kiran = Values

9. Features of Map

1. Stores data in key-value pair format
2. Keys must be unique
3. Values can be duplicated
4. Used for quick searching using key
5. Does not use index like List

10. Example of Map

$$\{101 = \texttt{"Ravi"},\ 102 = \texttt{"Anu"},\ 103 = \texttt{"Kiran"}\}$$

Observe:

- Each student is identified using a roll number
- Roll number acts as a key
- Name acts as a value

11. List vs Map

Feature	**List**	**Map**
Stores	Values only	Key-Value pairs
Duplicates	Allowed	Keys not allowed
Access	By index	By key
Example	Student names	RollNo $\rightarrow$ Name

12. Where is Map Used?

Real-World Applications of Map:

- Student database
- Phone contacts
- Banking account details
- Product inventory
- Employee management
- Login systems

13. Syntax of Map

General Syntax:

```
Map<KeyType, ValueType> variableName = new
    ClassName<>();
```

Example:

```
Map<Integer, String> students = new HashMap<>();
```

Breakdown:

- `Map` → interface
- `<Integer, String>` → key type, value type
- `students` → variable name
- `HashMap` → implementation class

14. Why Write Map Instead of HashMap?

Recommended Style:

```
Map<Integer, String> students = new HashMap<>();
```

Why?

- More flexible
- Easy to change implementation later
- Follows good Java programming practice

Example: Today:

```
Map<Integer, String> students = new HashMap<>();
```

Tomorrow:

```
Map<Integer, String> students = new TreeMap<>();
```

15. Common Methods of Map

Important Map methods:

- `put()`
- `get()`
- `remove()`
- `containsKey()`
- `containsValue()`
- `keySet()`
- `values()`
- `entrySet()`
- `size()`
- `clear()`
- `isEmpty()`

16. put() Method

Purpose

Used to insert a key-value pair into the Map

Example:

```
Map<Integer, String> students = new HashMap<>();

students.put(101, "Ravi");
students.put(102, "Anu");
students.put(103, "Kiran");
```

Conceptual Output:

$$\{101 = \texttt{"Ravi"},\ 102 = \texttt{"Anu"},\ 103 = \texttt{"Kiran"}\}$$

17. get() Method

Purpose

Used to retrieve a value using its key

```
System.out.println(students.get(102));
```

Output:

```
Anu
```

18. remove() Method

Purpose

Used to remove an entry using its key

```
students.remove(101);
```

Before:

$$\{101 = \texttt{"Ravi"},\ 102 = \texttt{"Anu"},\ 103 = \texttt{"Kiran"}\}$$

After:

$$\{102 = \texttt{"Anu"},\ 103 = \texttt{"Kiran"}\}$$

19. containsKey() and containsValue()

Examples:

```
System.out.println(students.containsKey(102));
System.out.println(students.containsValue("Ravi"));
```

Use:

- `containsKey()` → checks whether a key exists
- `containsValue()` → checks whether a value exists

20. size(), clear(), isEmpty()

Examples:

```
System.out.println(students.size());
students.clear();
System.out.println(students.isEmpty());
```

Use:

- `size()` → total number of entries
- `clear()` → removes all entries
- `isEmpty()` → checks if map is empty

21. Full Basic Program on Map

```
import java.util.*;

public class Demo {
  public static void main(String[] args) {
    Map<Integer, String> students = new
        HashMap<>();

    students.put(101, "Ravi");
    students.put(102, "Anu");
    students.put(103, "Kiran");

    System.out.println(students);
    System.out.println(students.get(102));
    students.remove(101);
    System.out.println(students);
    System.out.println(students.containsKey(103));
    System.out.println(students.size());
  }
}
```

22. Beginner Level Problem 1

Problem

Store 3 student roll numbers and names using Map and display them.

Expected Skills:

- Create Map
- Use `put()`
- Display Map

23. Beginner Level Problem 2

Problem

Store 3 product IDs and product names using Map.

Expected Skills:

- Understand key-value relationship
- Insert entries into Map

24. Intermediate Level Problem 1

Problem

Store 5 student records using Map. Search and display the student name for roll number 102.

Expected Skills:

- Use `get()`
- Search using key

25. Intermediate Level Problem 2

Problem

Store contact names and phone numbers using Map. Remove one contact and display the updated data.

Expected Skills:

- Use `put()`
- Use `remove()`
- Understand practical use case

26. HOTS Problem 1

Problem

Create a Student Result Management System using Map.

Requirements:

- Roll Number $\rightarrow$ Marks
- Add student marks
- Search marks using roll number
- Update marks
- Remove failed student record

27. HOTS Problem 2

Problem

Create a Phone Contact Manager using Map.

Requirements:

- Add contacts
- Search phone number using contact name
- Delete a contact
- Check whether a contact exists

28. Common Viva / Exam Questions

1. What is a Map in Java?
2. Why do we need Map?
3. What is a key-value pair?
4. What are the features of Map?
5. Can Map store duplicate keys?
6. Can Map store duplicate values?
7. Is Map a class or interface?
8. Is Map a child of Collection interface?
9. Name any two classes that implement Map

29. Key Takeaway

Final Understanding

A **Map** is used in Java when we want to store data in the form of **key-value pairs** for better organization and meaningful searching.

Remember:

- Keys are unique
- Values can repeat
- Access is by key, not by index
- Very useful in real-world applications

Next: **ArrayList**

Thank You!

Stay Connected

Premanand S

Email:	premanand.s@vit.ac.in
Phone:	+91-7358679961
LinkedIn:	linkedin.com/in/premsanand
Instagram:	instagram.com/premsanand
WhatsApp Channel:	anandsDataX
Google Scholar:	Google Scholar Profile
GitHub:	github.com/anandprems